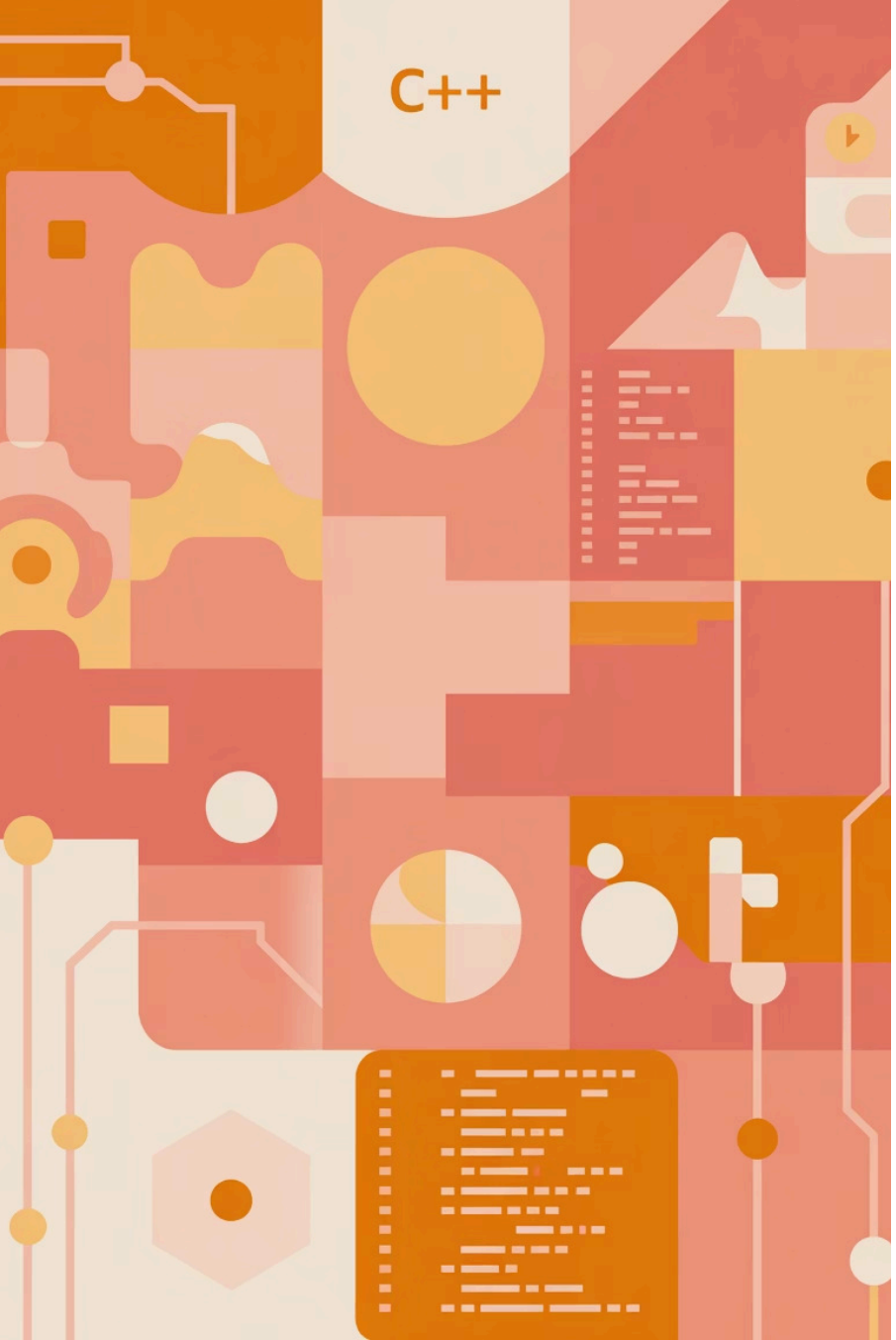

现代C++的发展道路

从哲学到实践

第二篇：零成本抽象——C++的设计哲学

一门语言的生存之道

零成本抽象

C++ 的核心设计哲学

在多数编程语言中，引入高层抽象往往意味着牺牲运行效率。然而，C++ 却独树一帜。

抽象的普遍代价

高级抽象通常会带来额外的计算开销，导致程序执行变慢。这是许多语言中权衡性能与开发效率的常见挑战。

C++ 的独特之处

C++ 的设计目标是让抽象代码能够编译出与手写 C 甚至汇编代码相媲美的执行速度，几乎不产生运行时开销。

本篇内容概要

我们将深入探讨“零成本抽象”的定义、其重要性，以及它如何深刻塑造了现代 C++ 的发展方向。

什么是抽象？

这里的抽象是指中高级编程语言为了帮助人类构建程序而创造出来的高层次概念。

C++ 聚焦的领域（如操作系统、数据库、游戏引擎、量化交易、高性能计算）往往需要构建巨大而复杂的软件系统。

没有抽象，人类将无法驾驭如此复杂的代码。开发大型软件时，我们不可能直接使用汇编语言或机器码，因此必须借助抽象来管理复杂性。

抽象的常见形式示例：

- 函数（Function）
- 结构体（Struct）
- 面向对象（Class, OOP）
- 模块（Module / 包）
- 泛型、模板（Template）
- 面向方面（AOP）

这些概念都是程序语言设计者为了让你"更容易构建大型程序"而引入的。它们本质上并不属于CPU和硬件指令集，在机器语言中本不存在。

零成本抽象的含义

使用高级抽象不会让程序变慢。这就是"零成本"的含义。在C++的每次演化和决策中，这一原则始终被优先考虑。

具体而言，希望达到这样的效果：

当用这些抽象时，不会额外引入运行时开销。

如果不用抽象，就不会产生任何额外代价

你自己手动实现也不会更快

实例：多态的零成本

如果你用 C 语言手动实现一个带有多态特性的结构体（例如通过函数指针模拟虚函数），它的性能不会比 C++ 原生的虚函数机制更高。C++ 的多态实现已经是"你自己实现也不会更快"的程度。

而如果在 C++ 中你不使用类的多态特性（也就是不定义虚函数），那么类的对象布局就不会包含虚表指针，也就没有任何额外的运行时开销。此时这个类的成本与 C 语言中的结构体 struct 完全相同。

零成本抽象的实现方法

编译期计算 / 展开

利用模板和静态多态，在编译时完成计算和代码展开，避免运行时开销。

按需付费原则

如果不使用某个特性，就不会为此承担任何运行时或内存成本。

直接硬件映射

抽象层与底层硬件紧密对应，确保不会隐藏或扭曲机器模型的性能特性。

编译优化

通过内联、条件分支预测（likely）和异常声明（noexcept）等机制，提升代码执行效率。

无全局运行时

不依赖统一的虚拟机或复杂的运行时环境，减少额外的抽象层和开销。

编译期模板推导展开以及编译期计算

在编译期完成计算和代码生成，运行时零开销。

模板推导和展开

C++ 的模板在编译期实例化，模板函数会被展开、内联，"模板"这一抽象在编译后就被"消失"了。

静态多态（CRTP等）

静态多态（CRTP 等）在编译期就确定了实际执行的函数。因此静态多态没有一般多态所需要的代价。

编译期求值（constexpr、consteval、fold expression）

在编译期间完成计算,运行时直接使用计算结果。

代码示例

```
template <typename... Ts>
constexpr double l2_norm(Ts... xs) {
    // fold: (x*x + ...)
    double sum = (... + (xs * xs));
    return std::sqrt(sum);
}

constexpr double r = l2_norm(3.0, 4.0); // r = 5.0 (编译期完成)
```

变量 r 的值在编译期就已经计算完成，运行时直接使用结果 5.0，无需任何计算开销。

按需付费：特性不使用就没有任何成本

C++ 的一个核心原则：只为你实际使用的特性付出代价。

不用异常

就没有异常开销

不用 RTTI

就没有 RTTI

不用多态

就没有虚表

不用，就等于成本不存在。

以 RTTI 为例

如果你在 C++ 代码里没有使用 RTTI（运行时类型识别），那么编译器就不会为你的程序生成任何 RTTI 相关的数据结构，也不会带来任何运行时开销。

不使用时

- 没用 dynamic_cast
- 没用 typeid
- 没开启需要 RTTI 的特性

结果

- 不会生成类型信息表
- 不会有类型名称字符串
- 不会增加额外的对象元数据
- 不会占用运行时内存

与硬件直接映射

C++ 的底层特性紧贴真实硬件：对象布局、内存访问、指针行为都与机器模型一致。语言不会插入 VM、GC 这类中间层，不会遮挡硬件细节，使你能直接控制内存、布局和性能。

C++ 的抽象不会强行插入一个"中间层"

（比如 VM、GC、自动内存模型等）

你写的代码大致能预测最终会生成什么样的机器指令

程序的性能特征与你的代码逻辑保持一致

（内存访问、缓存命中、对象大小、栈/堆分配）

这对'零成本抽象'很重要，因为抽象不会强制增加运行时负担，也不会妨碍你预测和优化实际的机器行为。

C++ 的设计哲学

> 抽象能帮你写得更容易，但不会妨碍你直接控制硬件。

与其他语言的对比

其他语言（如 Java、Python）

- 所有对象都必须堆上分配
- 内存由 GC 管理
- 类型布局由 VM 决定
- 运行时模型与你写的代码不一致

C++

- 刻意避免这些限制
- 直接控制内存分配
- 透明的对象布局
- 代码与硬件行为一致

为什么这对零成本抽象很重要？

如果语言的抽象层太厚，会导致：

- 无法准确控制性能
- 无法预测真正的对象布局
- 无法决定何时分配/释放资源
- 无法消除不必要的隐藏成本

无虚拟机、无垃圾回收，编译期优化手段

C++ 不依赖虚拟机、统一对象模型或垃圾回收器，也没有全局运行时调度器。语言层没有强制性的中间层，因此不会为抽象引入固有成本。

编译期优化

编译器可以基于语言的静态特性进行激进优化，包括：

- 内联函数，消除调用开销
- noexcept 帮助移除异常路径带来的额外检查
- [[likely]]、[[unlikely]] 改善分支预测
- 利用类型信息进行更高效的数据布局和优化

这些机制共同保证：

C++ 提供的抽象几乎不带运行时成本，不使用的特性不会产生任何额外代价。

C++的零成本抽象例子

C++ source #1

```cpp
#include <iostream>

using namespace std;

void f1(int a, int b)
{
    cout << a + b;
}

class MyClass
{
private:
    int mValue{};

public:
    MyClass(int v) : mValue(v) {}
    MyClass() = default;
    MyClass operator+(const MyClass& other) const
    {
        return MyClass(mValue + other.mValue);
    }
    int value() const { return mValue; }
};

void f2(int a, int b)
{
    MyClass oa(a);
    MyClass ob(b);
    MyClass oc = oa + ob;
    cout << oc.value();
}

int main()
{
    int a,b;
    cin >> a >> b;
    f1(a,b);
    f2(a,b);
}
```

x86-64 clang 17.0.1 (assertions) (Editor #1)

x86-64 clang 17.0.1 (ass -O2 -std=c++14

Output... Filter... Libraries Overrides Add new... Add tool...

Include libs

```asm
f1(int, int):
        add     esi, edi
        mov     rdi, qword ptr [rip + std::cout@GOTPCREL]
        jmp     std::ostream::operator<<(int)@PLT

f2(int, int):
        add     esi, edi
        mov     rdi, qword ptr [rip + std::cout@GOTPCREL]
        jmp     std::ostream::operator<<(int)@PLT

main:
        push    rbx
        sub     rsp, 16
        mov     rdi, qword ptr [rip + std::cin@GOTPCREL]
        lea     rsi, [rsp + 12]
        call    std::istream::operator>>(int&)@PLT
        lea     rsi, [rsp + 8]
        mov     rdi, rax
        call    std::istream::operator>>(int&)@PLT
        mov     esi, dword ptr [rsp + 8]
        add     esi, dword ptr [rsp + 12]
        mov     rbx, qword ptr [rip + std::cout@GOTPCREL]
        mov     rdi, rbx
        call    std::ostream::operator<<(int)@PLT
        mov     esi, dword ptr [rsp + 8]
        add     esi, dword ptr [rsp + 12]
        mov     rdi, rbx
        call    std::ostream::operator<<(int)@PLT
        xor     eax, eax
        add     rsp, 16
        pop     rbx
        ret
```

Output (0/0) x86-64 clang 17.0.1 (assertions) - 1385ms (70787B) ~5813 lines filtered Compiler License

https://godbolt.org/z/esWP7rPEd

本篇总结：零成本抽象的精髓

零成本抽象是 C++ 重要的设计哲学之一，确保了语言在提供高级抽象的同时，不牺牲运行时性能。

性能无妥协

C++ 的抽象不引入运行时开销，代码执行效率媲美手写C或者汇编。

多重实现策略

通过编译期计算、按需付费、内联，在编译期消除抽象成本。

开发者掌控力

提供强大的底层控制，允许开发者精确预测和优化性能。

下一篇我们重点关注

RAII —— 「让资源管理自动、确定、可靠、零泄漏」

我们将揭秘 C++ 如何在没有垃圾回收机制的情况下，通过 RAII 机制实现更确定、更高效的资源管理，让你写出永不泄漏的代码。